

Transika

Institutional System Architecture

Nontechnical overview first, then a technical deep-dive of the Transika/ platform: OTC desk, tGHS custody on Base, Ghana payouts, merchant and ops apps.

Scope

Transika Institutional only (Transika/ folder)

Audience

Business stakeholders -> then engineering

Transika Ltd · info@tran-sika.com · 2026-08-29

Contents

- A. Nontechnical overview
- 1-4 Layout, context, runtime, API domains
- 5-8 Auth, OTC, tGHS, Ghana payouts
- 9-10 Ops console, security & operations

A. Nontechnical overview

Transika Institutional helps regulated money-movement businesses trade foreign currency against Ghana cedis, hold value as tGHS (a Ghana-cedi stable token), and pay out to mobile money or bank accounts in Ghana — with Transika operations supervising sensitive steps.

What it is for

- OTC desk — quoted buy/sell of stablecoins against GHS (not a public exchange)
- tGHS treasury — digital Ghana-cedi balance for institutional partners
- Ghana payouts — send GHS to MoMo or bank beneficiaries after a trade or redemption
- Partner self-serve dashboard + optional API, with Transika ops controlling KYB and Live mode

Who is involved

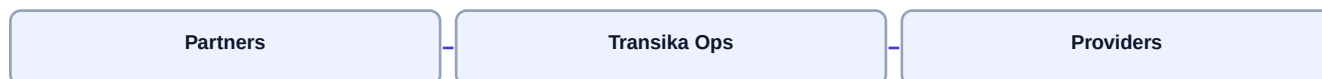


Figure A — Three audiences

- Partners — dashboard for trades, beneficiaries, tGHS redeem; API keys if enabled
- Transika ops — KYB, Live enable, quotes, mint with multisig, treasury monitoring
- Providers — Circle (wallets), Base (tGHS mint/burn), Transika Payout Service (Ghana MoMo/bank)

Typical partner journey

1. Sign up -> KYB review by ops -> product access
2. Start in Test mode (rehearsal); Live requires a separate ops enable
3. OTC: quote -> accept -> fund/proof -> ops confirms -> settle (tGHS or Ghana payout)
4. Redeem tGHS to a saved MoMo/bank beneficiary when local cash is needed

OTC desk

Human-supervised conversion. Quoted prices, checked funding, intentional settlement.

tGHS treasury

Cedi-denominated digital balance. Live uses on-chain tGHS; redeem pays out in Ghana.

Test vs Live

Test = safe practice. Live = real money, only after KYB + explicit Live enable.

Sections 1-10 below are the technical architecture for implementers. The overview above is enough for a business or ops briefing.

1. Monorepo layout (Transika/)

Everything in this document lives under Transika/: API, merchant app, ops admin, docs, and TGHS smart contracts.

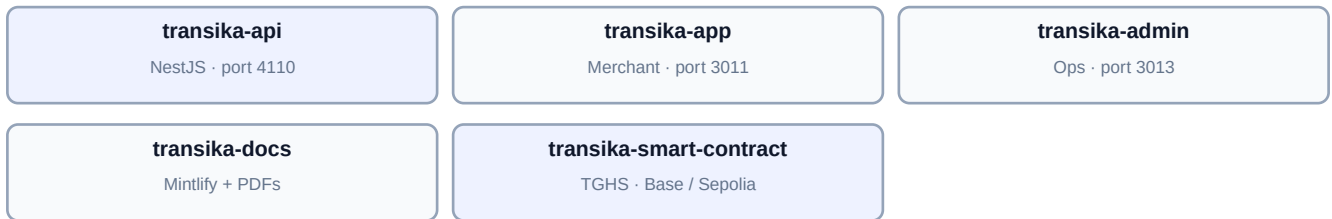


Figure 1 — Packages in Transika/

Internal docs: transika-api/doc/. Partner docs: transika-docs/. This PDF: transika-docs/scripts/generate-architecture-pdf.mjs.

2. System context

Partners and ops use Transika apps; the API orchestrates custody and payout providers. Merchants never hold Circle or Safe production secrets.

Actors

- Merchant users — JWT + OTP, team invites, OTC, beneficiaries, redeem
- Merchant systems — API keys (txk_live_ / txk_test_) with IP allowlists
- Ops staff — roles: admin, operations, treasury, compliance, viewer
- Treasury Safe owners — propose / co-sign mint and whitelist on Base

External systems

Circle

Developer-controlled SCAs for Live tGHS; signed webhooks

Safe

Multisig mint and wallet whitelist on Base

Transika Payout Service (TPS)

Primary Ghana MoMo/bank rail

Hubtel / Eganow

Legacy rails (rollback / name enquiry)

SMTP / ZeptoMail

OTP, invites, KYB, OTC alerts

Chainalysis KYT

Optional registration of OTC funding hashes

3. Runtime topology

Three frontends, one API process, one background worker, Postgres + Redis. The API runs migrations on boot; the worker does not.

Process view



Figure 2 — Runtime pieces

- Live API: /v1/* · Test API: /v1/sandbox/*
- Auth always uses Live /v1 so cookies stay on one path
- Apps host-align API hostname for first-party httpOnly cookies
- Sandbox: sbx-institutional*.tran-sika.com · Prod: institutional*.tran-sika.com
- Local defaults: API 4110 · merchant 3011 · admin 3013

4. Domain architecture (transika-api)

Modular NestJS. Each domain owns TypeORM entities. Schema changes via migrations only (synchronize: false).

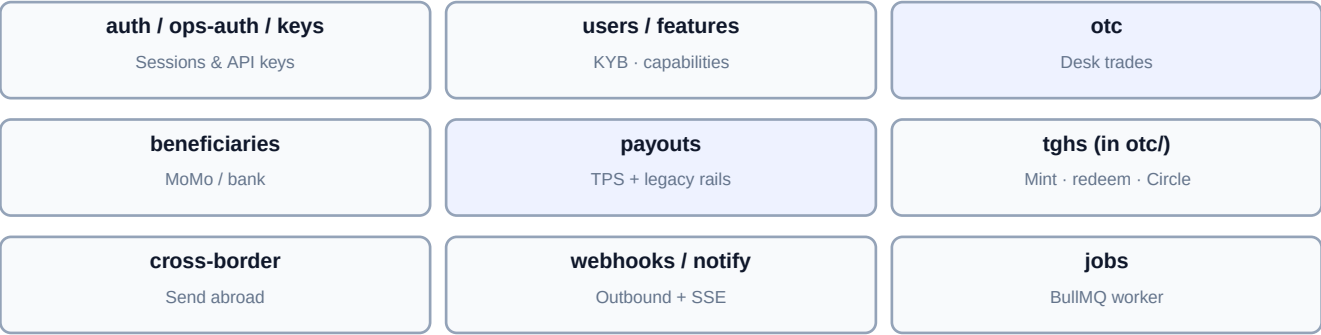


Figure 3 — Primary Nest modules

Background jobs

- webhook-deliver / webhook-delivery-sweep
- transika-pending-sweep (and legacy hubtel sweep)
- circle-tghs-redemption-sweep · tghs-redemption-burn

5. Authentication & entitlements

Merchant

- Email/password + OTP (skipped for Test Partners)
- httpOnly access + refresh cookies
- API keys: txk_live_ / txk_test_ · api:read/write · mandatory IP allowlist
- KYB pending until ops approve; Live separately enabled

Ops

- Ops JWT + OTP; OPS_SEED_EMAIL / OPS_SEED_PASSWORD on boot
- Roles gate routes (e.g. treasury:read, compliance:write)

Features & capabilities

otc / api_access

Merchant-requestable

cross_border

Ops-grant only (Send abroad)

buy_crypto_enabled

Fiat->crypto OTC

payout_momo / payout_bank

OTC settle to MoMo/bank (often default off)

Not gated by payout caps

Creating beneficiaries and redeeming tGHS stay available even when OTC MoMo/bank settle is disabled.

6. OTC desk flow

Quoted desk workflow: rate -> accept -> fund -> ops confirm -> settle. Crypto->fiat pays a beneficiary via the payout rail; fiat->crypto credits tGHS / mint as configured.

Trade status (happy path)

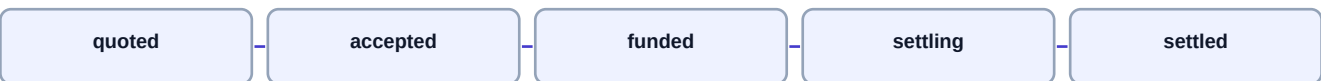


Figure 4 — OTC lifecycle

Fiat -> crypto

1. Quote (USDC/USDT + GHS) -> accept -> fund with proof/tx hash
2. Ops confirm-funded (manual desk check)
3. Settle credits tGHS / mint path

Crypto -> fiat

1. Partner pays assigned otc_deposit_wallet
2. Ops confirm -> settle to MoMo/bank beneficiary
3. PayoutService -> TPS -> webhook/sweep -> trade settled

Deposit wallets

otc_deposit_wallet is set only by a Treasury Safe owner. Quotes must use that address.

7. tGHS custody, mint & redemption

Test tGHS is a ledger balance. Live uses Circle SCAs. Live mint/redeem need the SCA whitelisted on TGHS (EOA on Sepolia; Compliance Safe on Base mainnet).

Live wallet states

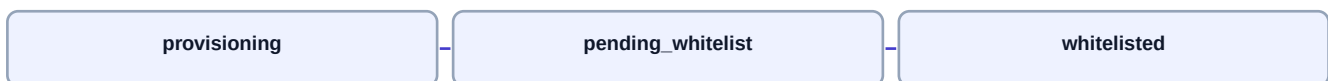


Figure 5 — Circle SCA lifecycle

Mint

1. Ops create mint request (+ funding evidence as required)
2. Safe owners propose and meet signature threshold
3. Execute on-chain; API reconciles; Live balance updates

Redemption path



Figure 6 — Redeem -> burn -> payout

- Live redeem auto-submits burn (no ops gate on burn)
- Balance reserved until burn confirms; burn_failed is recoverable
- Circle webhooks at POST /v1/webhooks/circle; worker sweeps delays

8. Ghana payout rail (TPS)

PayoutService talks to pluggable rails. With TRANSIKA_PAYOUT_ENABLED, active rail is transika (TPS). Otherwise simulated (legacy Hubtel/Eganow remain for rollback).

Institution catalog

- code — merchant networkCode (MTNGH, FIDELITY, ...)
- payout_service_code — TPS MoMo network / bank_code (MTN, TELECEL, ...)
- Seeded by migrations; managed in Admin -> Treasury -> Payout institutions

What we send TPS

transaction_id

Our payout_disbursements.id

reference

Merchant payout_service_reference (e.g. Kaprex_Transika)

network / bank_code

Institution payout_service_code

ghs_amount

Disbursement amount

Webhook

{API_PUBLIC_URL}/v1/webhooks/transika/payout/{TRANSIKA_PAYOUT_WEBHOOK_SECRET}

Idempotency

- One disbursement per (source_type, source_id); retries reuse same id
- Called from: tGHS redeem after burn_confirmed; OTC crypto->fiat settle; ops retry

9. Ops console & treasury

transika-admin uses ops JWT cookies against the same API. Mode toggle scopes desk data; auth always hits Live / v1.

Users / Settings

KYB, Live, caps, features, IMTO, OTC wallet, TPS reference

OTC

Quote, funding, settle, proofs

Mint

Requests + Safe propose / sign / execute

Treasury

Reserve view, payouts, institutions, failed payouts

Wallets / Compliance

Circle SCA + whitelist / Safe

Staff / Test partners

Invites + sandbox merchants

Admin Docker

Next standalone images must COPY public/ and .next/static (public after standalone). Brand PNGs are also webpack-imported into /_next/static/media.

10. Security & operations

Controls

- CSP + HSTS + CORP/COOP on apps
- API-key IP allowlists fail closed
- Webhook path secrets (TPS / Hubtel / Eganow)
- Circle webhook signature verification
- Ops kill switches: capabilities + Live enable

Inbound webhooks

TPS

POST /v1/webhooks/transika/payout/{secret}

Circle

POST /v1/webhooks/circle

Hubtel / Eganow

POST /v1/webhooks/{provider}/payout/{secret}

Environment bring-up

1. Postgres + Redis; DATABASE_URL / REDIS_URL
2. API_PUBLIC_URL, ALLOWED_ORIGINS, JWT, SMTP
3. Enable TPS + register webhook with TPS team
4. Build, migrate, start API + worker
5. Deploy apps with NEXT_PUBLIC_API_URL=https://...-api.../v1 (not frontend:4110)
6. Seed ops user; approve merchants; set institution codes & payout references

Related docs

- transika-api/doc/system-guide.md, architecture.md, payouts.md
- transika-api/doc/tghs-custody.md, otc.md, authentication.md

Implementation under Transika/transika-api/src and the apps is the source of truth if docs diverge.